Rectangular portfolios and exact kernels for subexponential SAT contraction

A third review of the combined research frontier

Research memorandum prepared for Vladimir Reshetnikov

27 October 2026

Abstract

The latest supplied packets already improve substantially on the previous iteration. This review compares against that combined frontier, rather than reusing obsolete baselines. Composing the genuine rectangular matrix backend with explicit four-plan tree portfolios gives a new general-degree normalized time coefficient $30.491276185937\ldots$ and space coefficient $28.853020941407\ldots$, versus the strongest comparable incoming pair $30.891018920366\ldots$ and $30.753912832055\ldots$. A separate polynomial-space composition gives $46.071078195848\ldots$, versus $46.184845586730\ldots$. For degree at most $s^{.51}$, coefficients are $21.139416167998\ldots$ in polynomial space or $14.502837849049\ldots$ in both cached time and space; degree at most $s^{1/12}$ gives cached time and space $12.015609290141\ldots$. These coefficients multiply $s/\sqrt{\lceil \log_2 s \rceil}$ in a base-two exponent; they do not improve the logarithmic SAT order.

An exact separable matching-potential representation reduces update work from the maintained dense $O(MQ^2 + VQ)$ baseline to $O(MQ + VQ)$ rational operations, preserving every candidate value and tie. A new mixed-root implementation extends coverage of an *inherited* projective coefficient identity to orders three and five, with paid preparation and adverse benchmark cases retained. Neither component is a complete SAT solver.

The independent Anthropic development is now public. Published repository CI for the exact linked source tree reports acceptance of both advertised theorem statements by Lean and nanoda. This materially supersedes the previous "unpublished formalization" description. We inspect source, statement scope and CI evidence, but do not claim a local kernel reproduction or transfer that verification to the refinements in this report.

Contents

1 Results and comparison rules . . . 3
2 The newly public Anthropic formalization . . . 4
3 Normalized interfaces and proof dependencies . . . 7
4 Review of the new packet frontier . . . 9
5 Rectangular support portfolios . . . 11
6 A fully paid generic cached schedule . . . 14
7 The strengthened four-plan certificate . . . 16
8 Streamed and degree-sensitive compositions . . . 17
9 Exact matching potentials in linear label work . . . 20
10 A new mixed-root implementation of an inherited identity . . . 24
11 Barriers and unresolved interfaces . . . 26
12 Evidence levels and independent checks . . . 27
13 Reproducibility and artifact map . . . 28
14 Conclusions . . . 29
A Source inventory and citations . . . 30

1 Results and comparison rules

Three different kinds of progress survive a comparison against the newly combined source corpus: smaller explicit contraction coefficients, a lower arithmetic order for exact matching-potential updates, and a wider practical implementation of an already known coefficient identity. The independent public formalization is a fourth development, concerning proof status rather than the novelty or implementation of our refinements. Keeping these claims separate prevents a real improvement in one category from being mistaken for a result in another.

Set

$$\lambda = \lceil \log_2 s \rceil , \qquad E(s) = s/\sqrt{\lambda}, \qquad \kappa = \frac{\pi\sqrt{2}}{\ln 2}. \tag{1}$$

Here s is the normalized number of Boolean wires, and $\mathcal{L}$ is the input-description length. A time coefficient C denotes

$$T(s, \mathcal{L}) \le \operatorname{poly}(s + \mathcal{L})\, 2^{(C+o(1))E(s)}.$$

A displayed space coefficient has the analogous meaning; "polynomial" means no such subexponential cache is required. A decimal rounded upward is a safe *eventual* coefficient, not a finite-size runtime prediction. Every coefficient in this report is resource- and premise-qualified.

Route	Time	Space	Root degree
Incoming adaptive flags	46.184846	Polynomial	$s^{.9}$
New flags plus strict schedule	46.071079	Polynomial	$s^{.9}$
Incoming three-plan tree	30.891019	30.753913	$s^{.9}$
Rectangular composition, old schedule	30.629498	29.985756	$s^{.9}$
Three plans, retuned	30.614394	29.966735	$s^{.9}$
New four-plan portfolio	30.491277	28.853021	$s^{.9}$
New degree-sensitive flags	21.139417	Polynomial	$s^{.51}$
New degree-sensitive cached companion	14.502838	14.502838	$s^{.51}$
Incoming very-low-degree cached	13.379650	13.253025	$s^{1/12}$
New four-plan low-degree route	12.015610	12.015610	$s^{1/12}$

Table 1: Explicit compatible certificates, not global optima. Values in this summary are rounded upward at six places. Cached rows use subexponential space. Each degree bound permits the harmless ceiling convention.

The old iteration-two generic coefficients 55.331622… in polynomial space and 35.070614… with caching are no longer the strongest comparison points. Reporting a gain only against them would conceal advances in the newly supplied work. Likewise, low-degree rows must not be compared as if they applied to every instance accepted by the generic branch. The prior lower-memory and degree-sensitive options remain available when their assumptions and resource tradeoffs are useful.

1.1 What this iteration contributes

1. **A combined rectangular support certificate.** The backend, three-plan menus and paid recurrence are credited to incoming work. This iteration constructs their combined finite support vectors, exhaustively checks 890,162 three-plan support tuples with exact integers, proves the resulting local estimate, and gives an explicitly feasible rational schedule. A bounded explicit four-plan extension then checks all 73,298,242 quadruples and strengthens the result again. On the unchanged inherited schedule the improvement already holds, isolating the composition from the additional retuning.

2. **Compatible streamed and low-degree compositions.** The adaptive flag lemma replaces weaker local separators inside an existing strict schedule. The new rectangular coefficient also improves a separately stated very-low-degree recurrence. Neither statement uses a free child-entry oracle.
3. **A new factorization of matching products.** The exact potential becomes a scalar times row factors times a sparse completed-edge matrix. Maintaining row sums reduces candidate-update work by a factor of Q in the arithmetic model, compared with the strengthened maintained-dense baseline.
4. **An implementation extension, not a new identity.** Mixed order-three and order-five filtering is already covered by the inherited projective theorem. The new code supplies this case in one fixed quotient ring, with an independent coefficient oracle, root validation, Gray traversal and matched component measurements.
5. **A corrected formalization assessment.** The linked public source and terminal successful verification job establish substantially more than the unpublished-proof claim available in the previous review. The present report records what was checked, by whom, and at which exact tree.

1.2 What does not follow

The inherited density-sensitive envelope remains

$$\mathrm{poly}_k(n,m)\,2^{O_k\left(\frac{n}{\sqrt{\log n}}\min\{\log(2+m/n),\log\log n\}\right)}. \tag{2}$$

The schedule changes an explicit normalized constant, not the power of $\log n$. A wire-size constant cannot be inserted unchanged into a bound in SAT variables n; the encoding's wire count, degree and description overhead must first be paid. Nor is there a common published leading constant allowing a numerical comparison with the independent crossing-decomposition algorithm.

A component arithmetic bound is not a bit-operation bound unless operand lengths and setup are charged. A component wall-time ratio is not a SAT speedup. A source scan is not kernel checking. Published kernel acceptance of specified statements is not automatic verification of every prose claim, cost-model explanation or new refinement. These distinctions will be maintained throughout.

2 The newly public Anthropic formalization

The user's linked revision is a release of the independent paper *and full Lean development*, not merely a revised informal assertion. At the observed 03:26 UTC snapshot, PR #31 in `anthropics/formal-math` is merged and the relevant public verification job is terminal success. The job log reports acceptance by both Lean's default kernel and the independent nanoda checker, with Comparator accepting the two advertised theorem configurations. The earlier description "Lean proof unpublished" is therefore superseded. This report did not execute Lean, nanoda, an installer or a project script locally. [3, 5]

2.1 Exact revision identity and chronology

The requested head is

`1a6eee293b9e00cb432eef6018a21f523623a9af`.

Git API tree identities show that it, GitHub's synthetic PR merge and the final merge contain exactly the same repository tree. The common tree is

`ab1069cc17ae5088e175ec37fa16fe8bd273753d`.

Thus the inspected successful job concerns the identical source bytes, not merely a nearby commit. The chronology is:

Event on 27 October 2026	UTC
Requested head commit	02:45:07
PR created	02:46:32
Verification job started	02:46:47
GitHub review approved	03:02:45
Previous report delivered	03:06:14
PR merged	03:07:22
Verification job completed successfully	03:22:05

The previous delivery predates the merge but *postdates public PR creation*. It would be incorrect to infer from merge time that the proof must have been unpublished at that delivery. The accurate correction is that public availability and verification were not established in the prior review and are established now. The GitHub approval contains no substantive review text; it should not be treated as an independent human audit of all mathematics.

The final merge is `16a5ce41304fcdaea4e0be7c132e1c361dc26da5`; the CI synthetic merge is `80ee23547e985fd4c7fc71fbc6910af85207bcf3`. The PR adds 1,063 changed files and 153,191 lines, with no deletions. Of these, 1,062 files are under `eth/`; the remaining change adds the root README entry. Exact API evidence, commit equivalence and the fetched log are retained locally. Primary links appear in the references.

2.2 The checked theorem statements

`MainResults.lean` imports Mathlib only and provides definitions of the claims. `Solution.lean` proves two closed theorems, with no additional theorem hypotheses.

ETH statement. `MainResults.ethIsFalse` has type `MainResults.ETHIsFalse`, the negation of $0 < s_3$. Here s_3 is the infimum of nonnegative exponents permitting deterministic finite-alphabet Mathlib multi-stack Turing-machine decision of 3-SAT in $O(2^{\delta n})$. The file makes the intended encoding and time notion explicit. This is a definite mathematical proposition, not merely an informal claim attached to a certificate name.

Uniform counting statement. `MainResults.mainTheorem` has type `MainResults.MainTheorem`. It asserts one deterministic machine and one natural constant C, chosen *before* k and the input formula, that count satisfying assignments within

$$\left\lfloor 2^{C \min\left\{ \frac{km}{\sqrt{\log_2 m}}, \frac{k^2 n \log_2 \log_2 n}{\sqrt{\log_2 n}} \right\}} \right\rfloor \tag{3}$$

steps. The explicit premises are $k \geq 1$, $m \geq 2$, $n \geq 4$, clause width at most k, no duplicate clause-list, and occurrence of all n variables. Thus this is uniform in k, not only a separate family of machines and constants for fixed k. Its constant is not numerically specified. [4]

The input includes k and n in unary and fixed-width literal slots. The output is the count in $n+1$ binary bits. Every stack alphabet is required finite, not only the input and output alphabets. Clauses are lists; an empty clause, repeated literals or opposite literals within one clause are allowed. Reordering a clause's literals can create a different list, so the no-duplicate condition refers to the formal representation rather than a quotient by logical equivalence. The small-size and occurrence conditions are explicit conventions, not unstated universal assumptions. Handling arbitrary presentations requires the corresponding preprocessing argument.

2.3 What the public verification establishes

The successful job is `verify (eth)`, run 40440389861, job 121979112896. Its fetched log records the following events, which are stronger evidence than a README assertion:

- the Lean build completed successfully with 9,768 jobs;
- both named results were exported from `Solution`;
- nanoda accepted the solution;
- Lean's default kernel accepted the solution;
- Comparator accepted its declared configuration.

The configuration names exactly the two main results, allows only `propext`, `Quot.sound` and `Classical.choice`, and enables nanoda. The CI script separately rejects an allowed-axiom list containing anything outside those three and forces nanoda on. Both `lake build` and the Comparator step are independently marked successful in the job-step API. These are statements about *published repository execution*, not a local independent reproduction by this review.

The only emitted placeholder warnings are the two intentional declarations in `Challenge.lean`. The solved theorem entry point does not import that file. Keeping challenge placeholders separate from solution dependencies matters: counting lexical placeholders anywhere in a repository is neither a sound rejection test nor a proof of cleanliness of the exported theorem environment.

2.4 Static inventory and the semantic boundary

The downloaded exact-commit archive has SHA-256

`d3e7da579f14747a7991003a863f6f88685f0f8a88e6777ca51863970143b07b`.

All extracted `eth/` files match their Git blob hashes. A lexical scan of 1,052 Lean files, removing nested comments, strings and line comments, finds only the two challenge placeholders. It finds no axiom declarations, `unsafe`, `native_decide`, `run_elab`, `implemented_by`, `extern` or `csimp`. A static import traversal from `Solution`, recognizing public, private and `import all` forms, reaches 810 internal source modules plus Mathlib/Batteries and does not reach `Challenge`. These counts are source-inspection facts, not kernel-level dependency extraction.

The released paper and the authenticated prior ProveIt copy both have 49 pages, but their PDF bytes differ. Direct extraction comparison finds Theorem 1 and its absolute-constant qualification unchanged after header and whitespace normalization. The entire printed Lean statement listing is unchanged after additionally removing comments. This is text-comparison evidence, not a compiler check. The release adds the public proof link and revises checking history, explanatory comments, provenance and editorial material; it is not a new asymptotic theorem. Its appendix prints the

trusted statement file and explicitly assigns human readers the task of deciding whether the definitions express the intended mathematics. Its historical statement that the development was AI-written and AI-reviewed, with no known person having checked it at writing, is not overturned by treating a short approval as comprehensive mathematical review. The paper's additional historical leanchecker runs were not independently verified here.

`proof/PaperIndex.lean` records departures from the paper's step-by-step derivation: polynomial-power rather than constant-factor alternation overhead, explicit table materialization and budget padding, cost-model conversions, different recursion induction, and intermediate decision/counting distinctions. Those differences are not silently erased by the final theorem. Importantly, the final checked statement counts steps of the specified finite-alphabet machine, rather than assuming an unbounded-cost internal operation model is free.

The standard simulation and encoding connection to the paper's word-RAM discussion is described in comments but is not itself part of the two final formal statements or their proofs. Kernel acceptance validates the given formal propositions; semantic correspondence to the intended complexity-theoretic assertion remains a separate audit. [6, 4] This is a precise boundary, not a reason to misdescribe the public formalization as absent.

2.5 Consequences for the present research

The proof-status comparison changes materially. The asymptotic rate does not improve merely because the proof becomes public: for fixed k, the displayed general rate remains $2^{O_k(n \log\log n/\sqrt{\log n})}$, with the clause-sensitive alternative in (3). The user-provided manuscript already advertised this order. There is no explicit common leading constant or shared implementation for a numerical comparison with our normalized coefficients.

The release does not establish a practical SAT runtime, polynomial-space bound, polynomial-time algorithm or resolution of P versus NP. It does not formalize our new support menus, schedules, matching factors or Python kernels. Conversely, the conditional interfaces used below should not be presented as weakening the independent checked theorem: these are distinct developments with different formal coverage.

3 Normalized interfaces and proof dependencies

Our theorems refine the contraction architecture in the supplied ProveIt packets. The normalized interface allows at most s Boolean wires, maximum root degree $\lceil s^{.9} \rceil$ for the generic branch, polynomially many vertices and polynomial tensor-description length. Root precision and truncation lengths are at most $O(s)$. Original admissible entries can be evaluated in

$$\mathrm{poly}(s + \mathcal{L})\, 2^{O(s^{.9})}$$

time and polynomial space. Entries are represented at immutable creation precision; reducing a displayed residue does not authorize redefining an entry later at a more convenient precision.

Arithmetic occurs in a free finite quotient of the form

$$(\mathbb{Z}/2^p\mathbb{Z})[U, V]/(f(U), V^\ell), \tag{4}$$

with polynomial module rank, $\deg f = O(\log s)$ in the asymptotic interface, and polynomially bounded coordinate bit lengths. This ring can have zero divisors and nilpotents. A field algorithm cannot simply be invoked as if all nonzero elements were invertible.

The following interfaces remain inherited:

1. exact deformation and coefficient recovery;
2. full slope-four source/target-local modes and their integrality;
3. exact list indexing and weighted entropy estimates;
4. nearly lossless asymmetric packing, including degree and precision gates;
5. recursive child-factor construction and affordable entry access;
6. immutable creation precision and valid Hermite reconstruction;
7. the SAT-to-normalized-contraction reduction, including density and size accounting.

The new finite lemmas and resource sums are proved relative to these interfaces. Static review of all new packets did not replace a full independent proof of the original contraction development. Published Lean/nanoda checking of the independent Anthropic source does not discharge these imports.

3.1 Weighted entropy and paid recurrences

At a call with mass M and retained taper u, define

$$K = \kappa M/\sqrt{u}.$$

For fixed directed edge coefficients z on an r-group partition, the imported weighted-entropy statement gives

$$\sum_e z_e h_e \le \left(\frac{\|z\|_2}{r} + o(1)\right) K, \tag{5}$$

uniformly on the active schedule. The h_e are exact mode-list logarithms. When a finite portfolio of support vectors is used, the maximum of finitely many error terms remains $o(1)$; no unbounded or data-dependent family is being silently inserted.

For a cached evaluator, constructing singleton group tables requires actual recursive child contractions. The common incident-pair coefficient is

$$\alpha_r = \frac{\sqrt{2(r-1)}}{r}.$$

If local aggregation has coefficient b_r, the paid recurrence is

$$T_d \le \mathrm{poly}(s)\left[2^{(\alpha_{r_d}+o(1))K_d} T_{d+1} + 2^{(b_{r_d}+o(1))K_d}\right]. \tag{6}$$

Its exponent contains child-call prefixes. Using only the root aggregation cost can understate time. In contrast, cached tables at different depths coexist additively in memory; their exponents do not add. Local storage needs its own input/output/workspace envelope.

A streamed evaluator revisits entries and pays a sum of path contributions, but can execute depth first in polynomial space. Its local coefficient cannot be substituted into a cached maximum, or vice versa. Polynomial factors per recursion level contribute lower-order exponent terms only after depth and description growth have been controlled.

4 Review of the new packet frontier

The intake is pinned to ProveIt commit `73dfb5e5ab6e91bd7375e5b97ef85d7e1cc8e81d`. A closing check at 03:33 UTC pins the final tree to `7de9d09fecd17de776d712835ecb2011205c531d`; the intervening changes relocate the research packets and add the Anthropic link, rather than introducing another proposal. All 443 files in the nine relevant final research directories match their authenticated arrival-archive Git blobs, with zero missing or mismatched files. Thus the mathematical intake remains the same after relocation. [1] Nine archive identities are recorded, eight new research packets plus the archived previous iteration. All incoming programs were treated as source data: no archived Python, Lean program, installer or third-party script was run. Newly authored checkers and kernels are distinguished from stored receipts supplied by others.

Incoming packet or route	Polynomial-space time	Cached time / space
Adaptive frontier	50.304689	31.252397 / 31.252397
Adaptive matrix partition	50.210435	34.325283 / 34.024178
Adaptive four	50.393726	32.915814 / 31.661707
Rectangular backend packet	53.977531	33.993891 / 33.993721
Adaptive bounds / tree frontier	46.184846	30.891019 / 30.753913
Round-three frontier	63.757356	37.930109 / 33.623041

Table 2: Incoming generic-degree certificates, shown approximately for comparison. Different rows contribute ingredients even when their isolated headline is dominated. This table is not a claim that every combination has been optimized.

4.1 Adaptive frontier and quartet selection

The adaptive-frontier packet supplies a structural improvement, not only a numerical retuning. For six nonnegative undirected edge weights on four blocks, let $a_j \ge b_j$ be the complementary pair weights in matching j. The best balanced binary pairing satisfies

$$F_4 = \sum_j (a_j + b_j) - \max_j b_j \le \sum_j a_j + \frac{2}{3}\sum_j b_j.$$

The squared six-edge norm is $13/3$, or $26/3$ on directed singleton edges. Its recursive disjoint-support ledger gives $g_4 = 26/3$, $g_6 = 22$, $g_8 = 110/3$, $g_{16} = 442/3$, and $g_r = 26(r^2-1)/45$ on powers of four. This improves the corresponding fixed-binary local norm by $\sqrt{13/15}$.

Its cached joint-objective selector pays three incident construction entropies together with the final rectangular operations, rather than optimizing the final triangle in isolation. The supplied covariance estimates give

$$\beta_9^2 = \frac{2489356}{6638625}, \qquad \beta_{12}^2 = \frac{4786024}{13540725}.$$

The root local work dominates its paid schedule at $31.252397\ldots$. Its same-coefficient space claim is subexponential, not polynomial. The code implementing finite contractions uses ordinary arithmetic and is not an implementation of the full asymptotic matrix backend or a SAT solver.

4.2 Matrix partitions and streamed trees

The matrix-partition packet uses the exact list-logarithm norm $\|h\|_2 \le (1+o(1))K/9$. After combining orientations, the undirected squared norm is at most $2K^2/81$. Across the 280 partitions into triples,

exact covariance values $1/2$, $1/28$ and $-1/21$ for equal, adjacent and disjoint edges yield final-triangle coefficient $\sqrt{57/175}$. Complete binary construction is more expensive and dominates at $\sqrt{916}/45$; ignoring that construction would produce an invalid headline.

Its streamed sixteen-leaf argument uses centered path loads, a permutation-invariant covariance matrix with eigenvalues 0, $33/56$ and $85/2912$, and a maximum-deviation bound. The resulting squared coefficient is $725/1344$. The supplied subset dynamic program has 14,827 states and 462,465 splits; it optimizes a finite balanced-tree maximum using integer ceiling logarithms. The additive rounding loss does not affect the leading exponent. These are coherent local proofs under the inherited gates, but the supplied output receipts were not independently executed as programs.

4.3 Adaptive four and the matrix model

The adaptive-four packet proves the sharp square-backend inequality

$$\min_{\text{matchings}} \max_{\text{merges}} \text{cost} \le \sqrt{244/75}\,\|w\|_2,$$

with a star-weight equality witness. For $7/3 \le \omega \le 3$, its parameterized undirected constant is $\sqrt{(\omega^2+4)/3}$, giving equal-four-cluster coefficient $\sqrt{(\omega^2+4)/24}$. The correction-vector sign conditions and full table-construction costs are accounted for. In its $8 \to 12 \to 8 \to 4$ schedule, a descendant paid-entry term dominates the root. An optional $\omega = 19/8$ specialization improves its cached pair to $32.856345\ldots/31.467483\ldots$, still dominated by the newer tree portfolio.

Combining that four-block inequality with the genuine rectangular backend gives a valid ablation with local square $[4v^2 + (2+w)^2]/24$. On the old four-block schedule this yields about 32.657863 time and 30.819243 space. This is not the headline: the three-plan tree composition improves both coordinates further.

4.4 Flags and finite tree portfolios

The adaptive-bounds packet's linear-flag proof identifies each nonzero difference with its unique flag layer. The affine-subspace intersection bound reduces to a homogeneous intersection bound; averaging maximizing leaves then gives the claimed Euclidean support norm. Its local optimality claim is correctly restricted to balanced binary trees and the entropy relaxation, not to every possible contraction algorithm.

The tree-certificate packet minimizes a complete plan cost over a small explicit menu. It uses $\min_j F_j \le \frac{1}{k}\sum_j F_j$ and enumerates fixed support tuples; it does not exchange a minimum with a maximum or assume arbitrary edge-coordinate permutations arise from vertex permutations. Every cached singleton factor is paid through α_r. Its storage proof uses the selected plan's actual operations, because an old size-only bound would not justify the improved space number.

Those exact interfaces make composition possible: replace the local operation support forms by the rectangular ones while retaining the plan menus and recurrence. Section 5 gives the new proof and independently authored exact certificate.

4.5 What is already inherited in practical kernels

The new adaptive-schedules packet already supplies matching-product increments in $O(MQ^2 + VQ)$ rational operations, compared with much weaker full recomputation. Its reported large speedups against that recomputation reference are not the correct baseline for our factorized implementation. Its common-denominator column scoring is also already known.

The prior iteration's positive-load sparse balancing is stronger than merely removing inactive row groups. It subtracts the load-zero baseline, visits only positive cells and chooses the least missing label while preserving exact ties. Likewise, the prior fused Cauchy transition kernel already lowers a complete table's multiplication count from $(5N^2 + N)/2$ to $N^2 + N$, with linear setup and one unit inversion in both routines. Neither should be reclaimed as new in this iteration.

For coefficient extraction, the strongest inherited centered baseline is anisotropic antipodal Gray traversal, not the weaker uniform step-two version. The inherited theorem already allows individual orders $\beta_j + 1$ and a minimum-exponent fixed coordinate. Its prototype's narrower implementation coverage creates an implementation opportunity, not mathematical novelty.

5 Rectangular support portfolios

The useful composition is to replace *every* operation in each complete tree plan by the genuine rectangular cost envelope, while preserving the recursive entry costs. It is not enough to accelerate only the final triangle. This section first gives the three-plan certificate, which isolates the effect of combining existing ingredients. Any larger certified finite menu uses the same lemma with its number of plans substituted.

5.1 The imported rectangular bridge

For sorted nonnegative logarithmic dimensions $x \geq y \geq z$, the incoming rectangular packet proves the bit-cost envelope

$$F(x, y, z) = \max\{d(x + y),\, x + vy + wz\}, \tag{7}$$

up to a polynomial factor in the normalized input parameters, where

$$\varepsilon = 10^{-5}, \qquad d = 1 + \varepsilon/2, \qquad v = \frac{47}{56} + \varepsilon, \qquad w = \frac{15}{28}.$$

The first branch cannot be dropped. Arithmetic exponent $2 + \varepsilon$ does not imply exactly quadratic work; the input/output term pays tables and is essential to the original certificate.

The external matrix facts needed are only dual exponent $\alpha > .3$ and square exponent $\omega < 19/8$. The primary paper *New Bounds for Matrix Multiplication: from Alpha to Omega*, arXiv:2307.07970, supplies the stronger inequalities $\alpha \geq .321334$ and $\omega \leq 2.371552$. No current-record claim is needed. [2] Fixed rational bilinear algorithms, tensor products at fixed base and constant-factor padding yield the required intermediate aspect ratios. Blocking the longest dimension handles unequal large axes. Dimension permutations are performed on the finite tensor decomposition, not by assuming a transposed cached program has free workspace.

To apply the backend to (4), the incoming bridge performs coordinate products over suitable odd primes, omits finitely many denominator primes, reconstructs the bounded integer coordinate products and then reduces in the actual ring. Polynomial module rank and coordinate precision keep this overhead polynomial. It does not divide by a nonunit in the power-of-two quotient. Workspace is a polynomial factor times the largest input/output array. We import this bridge rather than claim a new matrix algorithm or implement the complete asymptotic backend.

5.2 Exact integer support forms

Let $L_0 = 1{,}400{,}000$. Rearrangement gives an exact finite representation of (7) on unordered dimension logarithms: take all distinct permutations of

$$(1{,}400{,}007, 1{,}400{,}007, 0), \qquad (1{,}400{,}000, 1{,}175{,}014, 750{,}000) \tag{8}$$

and divide their maximum linear form by L_0. There are three supports in the first family and six in the second. This is an exact rational cost envelope, not a floating-log approximation.

For a binary merge of disjoint vertex sets A, B with complement O, the dimension classes are edges A–B, A–O and B–O. For a final triangle, they are the three block-pair classes. Edges internal to one part receive zero. Each operation therefore has nine integer support vectors in the space of undirected group-pair edges.

A plan constructs each final block by recursively splitting its ordered vertex list after $\lfloor k/2 \rfloor$ elements, then contracts the final three block tables. It has $r - 3$ binary merges plus one final triangle. The inherited menus are listed below; labels start at zero and order within each list determines the binary subtrees.

r	Ordered final blocks
9	$(0,1,2) \mid (3,4,5) \mid (6,7,8)$ $(6,2,5) \mid (4,7,0) \mid (1,8,3)$ $(5,2,8) \mid (4,6,1) \mid (0,7,3)$
13	$(0,1,2,3,4) \mid (5,6,7,8) \mid (9,10,11,12)$ $(12,10,1,6,4) \mid (9,0,3,8) \mid (5,2,7,11)$ $(0,8,11,12,1) \mid (7,10,4,3) \mid (5,2,6,9)$

For plan j, form V_j by taking and deduplicating the nine supports of all its operations. Write $g_{ab} = h_{ab} + h_{ba}$. Its complete operation cost is

$$F_j(g) = \frac{1}{L_0} \max_{v \in V_j} v \cdot g.$$

Choosing the minimum F_j pays block construction as well as the final contraction.

5.3 Finite support lemma

Lemma 5.1. *For a fixed menu of k complete plans on r groups, let*

$$Q_r = \max_{v_j \in V_j \ (1 \le j \le k)} \left\| \sum_{j=1}^{k} v_j \right\|_2^2 .$$

Under the weighted-entropy interface, minimum-plan work has local coefficient

$$b_r^2 = \frac{2Q_r}{k^2 r^2 L_0^2}.$$

The same coefficient bounds local stored arrays and backend workspace up to polynomial factors.

Proof. For every nonnegative g,

$$\min_j F_j(g) \le \frac{1}{k}\sum_{j=1}^{k} F_j(g) = \frac{1}{kL_0}\max_{v_j\in V_j}\left(\sum_{j=1}^{k} v_j\right)\cdot g.$$

For a fixed support tuple, put its coefficient on both directed orientations of each edge. Its directed squared norm is at most $2Q_r/(k^2L_0^2)$. Apply (5) and take the maximum over the fixed finite set of tuples. This yields $(b_r + o(1))K$ uniformly. It neither interchanges min and max nor invokes unrealizable permutations of edge weights.

Every intermediate table is an input or output of one of the counted operations. In (7), $d(x + y)$ dominates every pairwise dimension sum because x, y are the two largest logarithms and $d > 1$. Thus the selected maximum also bounds each stored array. The imported workspace envelope is bounded by the same largest input/output scale. □

5.4 The exact three-plan certificate

The independently authored standard-library checker reconstructs supports from the vertex lists, rather than trusting generated masks. It enumerates all tuples with unbounded integer arithmetic. The results are:

r	Supports per plan	Tuples	Q_r
9	57	185,193	246,255,154,308,036
13	89	704,969	519,754,477,218,816

As auxiliary reproducibility checks, the sums of all squared norms are

$$24{,}785{,}168{,}341{,}806{,}412{,}920 \quad \text{and} \quad 144{,}283{,}691{,}430{,}119{,}248{,}320,$$

and attaining witnesses are reconstructed coordinate by coordinate. A separately authored NumPy exploration obtained the same maxima before the standard-library replay. Normal and optimized Python outputs agree; validation does not depend on removable `assert` statements.

The local squares are

$$b_9^2 = \frac{2280140317667}{6615000000000}, \qquad b_{13}^2 = \frac{24170130079}{69316406250}, \tag{9}$$

so $b_9 = .587105113124937\ldots$ and $b_{13} = .590502138389288\ldots$. Exact mode-list cardinalities can select the plan without real arithmetic: compare products raised to the fixed integer support powers. These powers are large constants, but resulting bit lengths are polynomial. Alternatively, ceiling binary logarithms cause only $O(1)$ additive local error because the number of coordinates and coefficients are fixed.

The finite enumeration supports the numerical value in Lemma 5.1; it is not a proof of the inherited entropy estimate, rectangular bridge or upstream SAT theorem. The all-size theorem needs both the finite result and those explicit mathematical interfaces.

6 A fully paid generic cached schedule

The three-plan construction already improves both coordinates of the incoming cached frontier. Retain the square-backend tail at $r = 7$, with $b_7^2 = 632/1225$, and replace only the $r = 9, 13$ aggregation bounds by (9). This hybrid deliberately avoids needing a new tail certificate.

Theorem 6.1. *Under the generic normalized interface of Section 3, the three-plan rectangular composition admits*

$$T \le \mathrm{poly}(s+\mathcal{L})2^{(30.614393627106\ldots+o(1))E(s)},$$
$$S \le \mathrm{poly}(s+\mathcal{L})2^{(29.966734873162\ldots+o(1))E(s)}.$$

Safe eventual coefficients are 30.615 *and* 29.967. *The space is subexponential, not polynomial.*

6.1 Strict rational schedule and packing gates

Let $B = \lceil 6\log_2\lambda \rceil$ and stop at $H = \left\lceil s/\lambda^{3/4} \right\rceil$, using direct contraction below H. Set

$$r_0 = 9, \qquad f_0 = \frac{1577}{100000},$$
$$r_1 = 13, \qquad f_1 = \frac{2813}{250000},$$
$$r_d = 7 \quad (d \ge 2), \qquad f_d = \frac{1433}{300000}\left(\frac{4}{9}\right)^{d-2}.$$

Take $u_d = \lfloor f_d\lambda \rfloor$ and $a_d = \sum_{j\le d}(u_j + B)$. The leading cumulative fractions satisfy

$$A_1 = \frac{13511}{500000}, \qquad A_\infty = \frac{1781}{50000}.$$

The root old-degree gate is symmetric; subsequent old-degree gates are asymmetric. Exact sufficient inequalities are

$$9^{1577} < 2^{5000}, \qquad 13^{13511} < 2^{50000}, \qquad 7^{1781} < 2^{5000}. \tag{10}$$

They say $2f_0\log_2 9 < .1$, $A_1\log_2 13 < .1$ and $A_\infty \log_2 7 < .1$. These are strict fixed margins, not numerical equalities at a boundary. Fresh-side and rounding powers are bounded by

$$A_\infty(2\log_2 13 + 1) < \frac{1781}{50000}\left(2\cdot\frac{3701}{1000} + 1\right) < .3,$$

using the exact integer certificate $13^{1000} < 2^{3701}$. Newly created degrees have power at most $A_\infty < .9$.

Use padding tolerance λ^{-3}. Fixed positive power gaps absorb the $O((\log\lambda)^2)$ buffers, the active-call ratio $s/M < \lambda^{3/4}$ and the required polylogarithmic packing losses. To avoid circularly assuming the depth bound needed for packing, provisionally consider depths through $(3/4)\log_3\lambda + O(1)$. The tail taper there is at least

$$\lambda^{\eta+o(1)}, \qquad \eta = 1 - \frac{3\ln(9/4)}{4\ln 3} > 0.$$

Packing is uniformly valid over this range. Its tail child ratio is $2/7 + o(1) < 1/3$, which shows an active path cannot exceed the provisional range. Floors are negligible uniformly, and cumulative relative padding errors tend to zero.

Consequently

$$M_0 \le s, \qquad M_1 \le (1+o(1))\frac{2s}{9},$$

$$M_d \le (1+o(1))\frac{4s}{117}\left(\frac{2}{7}\right)^{d-2} \qquad (d \ge 2).$$

For retained $i < u_d$, the separation scale is $D = 2^{a_d - i} \ge 2^{a_{d-1}+B+1}$. Inherited precision is $O(s\lambda^2 2^{a_{d-1}})$, whose ratio to MD/u_d is $O(\lambda^{15/4}2^{-B}) = o(1)$. Thus the precision-six and weighted-entropy estimates remain uniform.

Leaves contribute $2^{O(s/\lambda^{3/4})}$, original entries contribute $\mathrm{poly}(s+\mathcal{L})2^{O(s^{.9})}$, and new packet entries contribute $\exp(s^{A_\infty+o(1)})$. Each is lower order on $E(s)$. Polynomial overhead per level gives only $O(\lambda \log \lambda)$ exponent bits. Immutable descriptions are retained; original-description dependence is not repeatedly re-exponentiated.

6.2 Time and space maxima

Write

$$K_0^* = \frac{\kappa}{\sqrt{f_0}}, \quad K_1^* = \frac{2\kappa}{9\sqrt{f_1}}, \quad K_2^* = \frac{4\kappa}{117\sqrt{f_2}}.$$

The tail K ratio is $(2/7)/\sqrt{4/9} = 3/7$ and

$$\frac{\alpha_7}{1-3/7} = \frac{\sqrt{3}}{2} > b_7.$$

Expanding (6) therefore gives the complete time maximum over

$$C_0 = b_9 K_0^*, \tag{11}$$

$$C_1 = \frac{4}{9}K_0^* + b_{13}K_1^*, \tag{12}$$

$$C_\infty = \frac{4}{9}K_0^* + \frac{\sqrt{24}}{13}K_1^* + \frac{\sqrt{3}}{2}K_2^*. \tag{13}$$

Exact rational enclosures give

$$29.966734873162 < C_0 < 29.966734873163,$$
$$30.614393627106 < C_1 < 30.614393627107,$$
$$30.491276185937 < C_\infty < 30.491276185938.$$

Thus C_1 governs time. Every deeper local table exponent is smaller than C_0, so C_0 governs space. This proves Theorem 6.1 under the imported interfaces.

The arithmetic verifier uses alternating Machin arctangent bounds for π, a positive $\mathrm{atanh}(1/3)$ series for $\ln 2$, and integer-square-root brackets. Floating-point discovery is not being used as the final inequality certificate.

6.3 Matched composition ablation

Keep the inherited fractions $(63/4000, 9/800, 77/16200)$ instead. The same new local support constants give

$$T:\ 30.629497100307\ldots, \qquad S:\ 29.985755335168\ldots.$$

Their tail paid limit is $30.512927432780\ldots$, below the middle-level maximum. Both coordinates already improve on $30.891018920366\ldots/30.753912832055\ldots$. Retuning is a further small gain, not the entire source of improvement. Percentage reductions in coefficients, if computed, are not percentages of measured runtime reduction.

7 The strengthened four-plan certificate

A bounded extension improves the three-plan ablation further while keeping its certified rational schedule fixed. At $r = 9$, use the four parallel classes of the affine plane on vertices $3x + y$, $(x, y) \in \mathbb{F}_3^2$:

$(0,1,2) \mid (3,4,5) \mid (6,7,8)$
$(0,3,6) \mid (1,4,7) \mid (2,5,8)$
$(0,4,8) \mid (1,5,6) \mid (2,3,7)$
$(0,5,7) \mid (1,3,8) \mid (2,4,6)$

Every pair of vertices is co-blocked exactly once. This motivates the menu but does not replace its complete operation certificate. At $r = 13$, retain the three inherited plans and append

$$(3,8,0,5,10) \mid (2,7,12,4) \mid (9,1,6,11),$$

the permutation $x \mapsto 5x + 3 \pmod{13}$ split into lengths $5, 4, 4$. Ordered block splitting remains part of every plan.

The exploration was bounded in advance: this one affine-plane menu at $r = 9$ and exactly three explicit fourth-plan choices at $r = 13$. The alternatives $2x + 1$ and $3x + 2$ gave squares 112953937728479/331240000000000 and 113129558528479/331240000000000, both larger than the chosen result. No global plan or schedule optimality is claimed.

Lemma 5.1 with $k = 4$ gives $b_r^2 = 2Q_r/(16r^2L_0^2)$. The exact certificate is

r	Supports per plan	Quadruples	Q_r
9	57	10,556,001	405,850,918,811,760
13	89	62,742,241	895,372,728,026,656

Hence

$$b_9^2 = \frac{563681831683}{1764000000000}, \qquad b_{13}^2 = \frac{27980397750833}{82810000000000}. \tag{14}$$

The approximate roots are .565285347085716 and .581280226306888.

The standard-library verifier exhausts all quadruples using exact Gram products:

$$\left\| \sum_i v_i \right\|_2^2 = \sum_i \|v_i\|_2^2 + 2 \sum_{i<j} v_i \cdot v_j.$$

This only accelerates arithmetic; it is not a relaxation. A maximizing tuple is reconstructed directly in coordinates. A separately authored verifier reconstructs operation sets with subset bit masks rather than the primary label map, permutes the plan enumeration order, and independently exhausts the same 73,298,242 tuples. Both normal and optimized runs agree. The factor two for directed orientations and denominator sixteen are explicitly audited.

Theorem 7.1. *Under the generic normalized interface, the four-plan algorithm with the frozen schedule in Section 6 has*

$$T \le \mathrm{poly}(s + \mathcal{L}) 2^{(30.491276185937\ldots + o(1))E(s)},$$

$$S \le \mathrm{poly}(s + \mathcal{L}) 2^{(28.853020941407\ldots + o(1))E(s)}.$$

Safe eventual coefficients are 30.492 *and* 28.854.

Proof. All packing, precision, degree, stopping and entry-cost gates are unchanged. Substitute (14) in (11)–(13); the singleton recursive costs remain unchanged. Exact intervals give

$$28.853020941407 < C_0 < 28.853020941408,$$
$$30.490561591676 < C_1 < 30.490561591677,$$
$$30.491276185937 < C_\infty < 30.491276185938.$$

Now the paid tail limit, not C_1, narrowly dominates time. Omitting it would understate the upper bound. Root local tables still dominate space by the selected-operation storage envelope. The infinite series bounds finite unrolled recursion paths; the algorithm still stops at the stated finite leaf threshold. □

No further taper transfer is needed for this certificate. A proposed tiny transfer from f_1 to f_2, preserving leading capacity, locally increases both relevant time terms at these parameters, so it is not advertised as an improvement. The schedule was deliberately frozen rather than extending an open-ended numerical search.

8 Streamed and degree-sensitive compositions

The cached construction must not displace a polynomial-space route when memory is constrained. The new flag bound and the rectangular packet's strict schedule are compatible because only the local streamed separator coefficient changes; the packing, degree, precision and depth gates remain the same.

8.1 Generic polynomial space

For $r = 4, 8, 16$, the incoming adaptive binary-flag lemma gives

$$b_r^2 = \frac{(r-1)(4r+10)}{9r^2}. \tag{15}$$

Use it at those alphabets and retain ordinary fixed separators $b_6 = \sqrt{24}/6$ and $b_5 = \sqrt{18}/5$. The strict schedule is

Level	r_i	f_i	Normalized mass m_i
0	16	$12499/10^6$	1
1	16	$12499/10^6$	1/8
2	8	2083/250000	1/64
3	6	107/20000	1/256
4	5	219/50000	1/768
Tail start	4	513/200000	1/1920

At tail depth j, multiply the taper by $(5/8)^j$ and mass by $(1/2)^j$. The full paid streamed sum is

$$C = \kappa \left[\sum_{i=0}^{4} \frac{m_i b_{r_i}}{\sqrt{f_i}} + \frac{m_5 b_4}{\sqrt{f_5}\left(1 - (1/2)/\sqrt{5/8}\right)} \right]. \tag{16}$$

It is $46.07107819584879\ldots$, with safe eventual coefficient 46.072, improving the incoming $46.184845586731\ldots$ certificate. Depth-first execution retains polynomial space. Replacing local paid costs by smaller valid bounds leaves the source schedule's strict feasibility margins untouched; this is a composition of proved ingredients, not a new flag lemma.

8.2 Very low root degree with three-plan caching

Assume the stronger root degree bound $\left\lceil s^{1/12} \right\rceil$. Fix $r = 13$ at all active depths, $A = 119/1000$, $q = 181/1000$, and set

$$u_d = \left\lfloor A(1-q)\lambda q^d \right\rfloor, \qquad a_d = \sum_{i \le d} u_i + (d+1)\left\lceil 6 \log_2 \lambda \right\rceil.$$

Stop at $H = \left\lceil s/\lambda^{2/3} \right\rceil$. The total leading taper is A, and both retained and newly generated degrees are at most $s^{A+o(1)}$, because $1/12 < A$. Every packing gate is dominated by

$$A(1 + 2\log_2 13) < 1,$$

with exact certificate $338^{119} < 2^{1000}$. On a provisional active path of depth $(2/3)\log_6 \lambda + O(1)$, the minimum taper is

$$\lambda^{1+(2/3)\ln q/\ln 6+o(1)},$$

which diverges since $q > 1/6$. Packing gives child ratio $2/13 + o(1) < 1/6$, closing the depth bootstrap. Buffers, precision and leaves are paid by the inherited low-degree theorem with strict margins.

With $K_0 = \kappa/\sqrt{A(1-q)}$ and $\gamma = (2/13)/\sqrt{q}$, recurrence (6) has coefficient

$$K_0 \max\left\{ b_{13}, \frac{\alpha_{13}}{1-\gamma} \right\}. \tag{17}$$

For the three-plan square in (9), exact enclosures show the first term strictly dominates. Therefore time and space both have coefficient $12.123986736154237\ldots$, safely below 12.125. Ancestor caches add in storage. This is a specialized result under the same stronger degree premise as the incoming $13.379649\ldots/13.253025\ldots$ pair, not a new generic-degree bound.

8.3 Final four-plan low-degree improvement

For the same root degree $\left\lceil s^{1/12} \right\rceil$, substitute the four-plan b_{13} from (14) and retune only q to 24/125. Keep $A = 119/1000$, so

$$f_0 = A(1-q) = \frac{12019}{125000}.$$

The total capacity certificate $338^{119} < 2^{1000}$, generated-degree bound, precision buffer and depth bootstrap are unchanged; in particular $q > 1/6$ still holds. Formula (17) has root-local term

$$C_{\text{local}} = 12.015609290141210\ldots$$

and whole paid-child-series term

$$C_{\text{children}} = 12.004596197275639\ldots < C_{\text{local}}.$$

Thus **time and space both have safe eventual coefficient** 12.016. Keeping the former $q = .181$ while lowering only b_{13} would leave the paid-child series dominant; the retuning is necessary for this root-local headline. This is another reason to show the complete recurrence rather than only the improved operation count.

8.4 Intermediate root degree in polynomial space

For root degree at most $\left\lceil s^{51/100} \right\rceil$, adaptive flags also improve the useful intermediate-degree branch. Replace the prior prefix's two $r = 15$ levels by two $r = 16$ levels, then use the following cumulative tapers:

d	r_d	A_d	$f_d = A_d - A_{d-1}$
0	16	1531/25000	1531/25000
1	16	1111/10000	2493/50000
2	8	357/2500	317/10000
3	6	81/500	12/625
4	5	1771/10000	151/10000
5	4	1911/10000	7/500
6	4	1999/10000	11/1250

For $d = 7 + j$, take $r_d = 3$ and $f_d = (391/40000)(3/4)^j$. The full taper budget is exactly 239/1000. With $m_0 = 1$ and $m_{d+1} = 2m_d/r_d$, the leading tail mass is $m_7 = 1/7680$, not the value from the old $r = 15$ prefix. Use (15) at $16, 8, 4$ and fixed balanced separators at $6, 5, 3$. The paid sum is

$$C_{.51} = \kappa \left[\sum_{d=0}^{6} \frac{m_d b_{r_d}}{\sqrt{f_d}} + \frac{(1/7680)(\sqrt{6}/3)}{\sqrt{391/40000}\left(1 - (2/3)/\sqrt{3/4}\right)} \right].$$

An independent rational verifier, using $\pi = 4(\arctan(1/2) + \arctan(1/3))$ rather than the primary Machin identity, proves

$$21.139416167998583798725141 < C_{.51} < 21.139416167998583798725142.$$

Thus 21.14 is a safe eventual polynomial-space coefficient.

The root degree gate is exactly $.51 + 8(.06124) = .99992 < 1$; depth-one rounding is $9(.1111) = .9999 < 1$. Later retained-degree and rounding gates are

$$.51 + A_d \log_2 r_d < 1, \qquad A_d(1 + 2\log_2 r_d) < 1.$$

The exact integer bounds $6^{200} < 2^{517}$, $5^{500} < 2^{1161}$ and $3^{200} < 2^{317}$ give fixed gaps. Since $A_{d-1} \le A_d$, fresh-side gates follow. Created degrees have power at most $.239 < .51$.

Keep $B = \lceil 6 \log_2 \lambda \rceil$, $u_d = \lfloor f_d \lambda \rfloor$, cumulative a_d with this buffer, and stop at $H = \lceil s/\lambda \rceil$. Provisionally consider depths through $\log_{10/7} \lambda + O(1)$. Their tail taper is at least

$$\lambda^{1+\ln(3/4)/\ln(10/7)+o(1)},$$

which diverges because $3/4 > 7/10$. Packing gives child ratio $2/3 + o(1) < 7/10$, validating the depth bound without circularity. Padding products are $1 + o(1)$; precision, leaves, packet entries and per-level polynomial overhead remain lower order. Depth-first evaluation uses polynomial space.

This improves the prior matching-degree coefficient $25.163765415229\ldots$. Since the premise includes degree $s^{1/12}$, it also dominates the older $23.666168807146\ldots$ low-degree polynomial-space route, but it does not replace the faster extra-memory low-degree branches.

8.5 A cached companion for intermediate degree

Under the same degree bound $\lceil s^{.51} \rceil$, use the four-plan $r = 13$ evaluator at every level with root taper $f_0 = 33/500$, total taper $A = 119/1000$, and tail

$$f_d = \frac{5353}{125000} \left(\frac{24}{125} \right)^{d-1} \qquad (d \geq 1).$$

Indeed $f_0 + f_1/(1 - 24/125) = A$. Exact packing certificates are

$$13^{66} < 2^{245}, \qquad 13^{119} < 2^{490}, \qquad 338^{119} < 2^{1000}.$$

They pay the root symmetric old-degree gate, later retained-degree gates and fresh-side/rounding gates respectively. Created degree stays below $.119 < .51$. Use the same buffer and $H = \left\lceil s/\lambda^{2/3} \right\rceil$ stopping rule as the very-low-degree cached branch; $2/13 < 1/6 < 24/125$ closes its depth and taper bootstrap.

Let $\beta = b_{13}$ from (14), $K_0 = \kappa/\sqrt{f_0}$ and $K_1 = (2/13)\kappa/\sqrt{f_1}$. The tail paid coefficient $\alpha_{13}/(1 - (2/13)/\sqrt{24/125})$ is strictly below β. The complete time maximum is therefore

$$\max\{\beta K_0,\, \alpha_{13} K_0 + \beta K_1\}.$$

Exact enclosures give

$$14.502837849049 < \beta K_0 < 14.502837849050,$$
$$12.172129729345 < \alpha_{13} K_0 + \beta K_1 < 12.172129729346.$$

Root tables also dominate storage. Thus 14.503 **is safe for both time and space**, a more-memory companion to the polynomial-space 21.14 route. This is a fixed corollary, not another schedule search, and the stronger degree premise remains essential. The exact checker includes negative controls for raising the root taper to .067 or total taper to .120.

9 Exact matching potentials in linear label work

The practical asymptotic improvement in this iteration is a representation change for the matching-based conditional-expectation packer. This is the matching-potential branch, not the asymmetric labeler used in the sharp contraction headline; a faster implementation of this branch is not automatically a faster implementation of every packing route. It reduces exact rational-operation count against the strongest supplied maintained-dense update, not just against full recomputation. The selected labels, candidate scores, trace and final loads remain identical.

9.1 State and factorization

Let a bipartite multigraph have M edges and V incident vertices. First-fit edge coloring partitions its edges into K matchings. There are Q labels, and η is the reference concentration parameter. Process all left vertices before all right vertices, matching the incoming sorted order of tagged vertices. Set

$$d = 1 + \frac{1}{\eta Q^2}, \qquad c = 1 + \frac{1}{\eta Q}, \qquad t = 1 + \frac{1}{\eta}.$$

For matching j, let U_j count edges with neither endpoint assigned, L_{ja} count edges whose left endpoint has label a and whose right endpoint is free, and C_{jab} count completed edges with label pair (a, b). With the incoming threshold exponent h_j, the conditional-product table factors exactly as

$$P_{jab} = s_j x_{ja} B_{jab}, \quad s_j = t^{-h_j} d^{U_j}, \quad x_{ja} = c^{L_{ja}}, \quad B_{jab} = t^{C_{jab}}. \tag{18}$$

Maintain

$$R_{ja} = \sum_b B_{jab}, \qquad S_j = \sum_a x_{ja} R_{ja}, \qquad \Phi = \sum_j s_j S_j.$$

Initially B is all ones, $R_{ja} = Q$, $x_{ja} = 1$ and $S_j = Q^2$. Store only nondefault B cells in a dictionary. At most one new cell arises per completed edge, so potential-state storage is

$$O\Big(KQ + \min\{M, KQ^2\}\Big) \tag{19}$$

rational values. This excludes the graph, the requested trace and the returned dense $Q \times Q$ load matrix. Producing the unchanged dense output still requires $\Theta(Q^2)$ writes and cells. Initialization includes $O(KQ)$ state writes plus the separately charged exact powers.

9.2 Left-vertex candidates

For an incident edge in matching j, the right endpoint remains unassigned. Assigning the current left vertex to label a replaces s_j by s_j/d and x_{ja} by cx_{ja}. Its common baseline change is

$$(s_j/d - s_j)S_j,$$

and candidate a receives the additional correction

$$(s_j/d)(c-1)x_{ja}R_{ja}. \tag{20}$$

The Q candidate corrections require $O(Q)$ operations per incident matching. Once a label is chosen, update s_j, x_{ja} and S_j in constant state work for that incident edge.

9.3 Right-vertex candidates

The left neighbor already has some label a. First replace x_{ja} by x_{ja}/c, producing the common change

$$s_j(x_{ja}/c - x_{ja})R_{ja}.$$

Candidate right label b receives

$$s_j(x_{ja}/c)(t-1)B_{jab}. \tag{21}$$

After selection, multiply B_{jab} by t and update R_{ja} and S_j in constant state work. An absent dictionary cell means $B_{jab} = 1$, not zero.

Proposition 9.1. *For the fixed left-then-right order and exact arithmetic, the factorized routine preserves every candidate potential, the least-label tie rule, the selected potential trace and the final label-pair loads of the maintained dense routine. Candidate-update work is $O(MQ + VQ)$ rational operations, compared with $O(MQ^2 + VQ)$ for the maintained dense table.*

Proof. Equation (18) is the product of the unassigned-edge, one-assigned-endpoint and completed-edge conditional factors, grouped by their state. Equations (20) and (21) are exact algebraic expansions of the affected products. A vertex has at most one incident edge in any one matching, so no update for that matching is double-counted. Summing the common part and candidate corrections therefore reconstructs every dense potential exactly. Equal candidate values give the same minimum and, with the same smallest-label rule, the same choice. Induction over processed vertices proves all later states, traces and loads agree.

Each edge has two endpoint updates and contributes Q candidate corrections at each. Scanning the label candidates at each vertex costs $O(VQ)$. Chosen-state updates are constant per incident edge. This gives the stated count. □

The proposition concerns update arithmetic. First-fit coloring can cost $O(MK)$, input sorting costs $O(V \log V)$, and initialization includes exact exponentiation. Fraction numerator and denominator growth remains charged; no constant-time-big-integer assumption is made. The factors have the same scale of bit growth as the original conditional products. Thus the algebraic improvement is meaningful without claiming that every rational operation has equal wall time.

9.4 Positive-support scoring in both assignment phases

The new factorization also composes with the *inherited* positive-support delta technique. This further refinement is an exact application to the matching potential, not a claim to invent sparse conditional expectations. Store both x and B with implicit default one; no full row-sum arrays are needed.

During the left phase, no right vertex has been assigned, so B is identically one. For incident matching j, put $s'_j = s_j/d$. After subtracting the label-independent unit baseline, candidate a has nonnegative delta

$$\delta_a = \sum_{j \text{ incident}} s'_j (c-1) Q (x_{ja} - 1).$$

Only nondefault x entries contribute. In the right phase, an incident edge has known left label a; first put $x'_{ja} = x_{ja}/c$, deleting the entry if it becomes one. Candidate b has delta

$$\delta_b = \sum_{(j,a) \text{ incident}} s_j x'_{ja} (t-1)(B_{jab} - 1).$$

Only stored cells in the relevant B rows contribute. All deltas are nonnegative. Conditional averaging recovers each full candidate potential from

$$\Phi_{\text{candidate}} = \Phi_{\text{current}} + \delta_{\text{candidate}} - \frac{1}{Q} \sum_{\text{labels}} \delta_{\text{label}}. \tag{22}$$

If fewer than Q labels have positive delta, choose the least absent label; otherwise minimize (δ, label) lexicographically. This preserves every label, tie and selected potential by induction. At completion, all x dictionaries are empty, and an independent exact final-potential check is

$$\sum_j s_j \left[Q^2 + \sum_{(a,b) \text{ stored}} (B_{jab} - 1) \right].$$

Let $X_j(v)$ count nondefault row factors at a left step, and $C_{ja}(v)$ count nondefault cells in a relevant row at a right step. Scoring work is

$$O\left(M + V + \sum_{v \text{ left}} \sum_{j \text{ incident}} X_j(v) + \sum_{v \text{ right}} \sum_{(j,a) \text{ incident}} C_{ja}(v) \right).$$

Finding the least absent label uses at most the touched-support size plus one probes, not an unconditional scan of Q labels. Each support is at most Q, so the worst case remains $O(MQ + VQ)$; the improvement is support-sensitive. At most M nondefault cells of each kind exist, giving $O(K + M)$ rational potential-state storage. Input, assignments, trace and the unchanged dense $\Theta(Q^2)$ load output remain separately charged, as do coloring, sorting, initial powers and rational bit costs.

If a diagnostic requests every candidate value, constructing that output still costs $O(VQ)$ and is paid. Ordinary benchmarks request identical assignments, matching data, thresholds, selected trace and dense loads from all three methods; sparse averaging deliberately avoids materializing an unrequested candidate vector. Full candidate equality is tested separately. The array-factorized routine remains the stronger comparator for this additional sparse step.

9.5 Correctness evidence and hardening

The authored implementation has a literal conditional-product oracle and an independently structured maintained-dense reference. Normal and optimized runs pass 696 exact four-method comparisons (oracle, dense, factorized and sparse), plus 32 larger maintained-reference comparisons. The oracle cases check 2,400 selected trace values and 6,000 candidate values. Coverage includes $Q = 1$, empty graphs, parallel edges, ties and $\eta = 1$.

An additional independent audit exhaustively enumerates complete labelings on 126 tiny matching instances, including 2,400 full labelings and 744 conditional candidate values. This is structurally different from replaying the same update equations. The checks are finite component evidence, not a formal proof, but they target exactly the values claimed preserved rather than merely the final load matrix.

9.6 Matched timing protocol

The benchmark compares with the maintained dense $O(MQ^2 + VQ)$ routine, not the literal oracle. The final comparator also caches its exact row sums; earlier provisional larger speedups against recomputed row sums are superseded. Five alternating and rotating repetitions follow an equal-output warmup. All three timed calls include graph coloring, exact initialization, selected trace, dense load output and each method's invariant checks. Sparse candidate averaging is symbolic, as in (22); unrequested full candidate vectors are not materialized. Generation is outside all measured regions. Raw wall and CPU samples, exact graph edges, interpreter version and implementation SHA-256 are preserved.

Fixture	M	Q	D ms	F ms	S ms	F/S wall	F/S CPU
One edge	1	1	0.070	0.073	0.053	1.38	1.39
One edge	1	2	0.177	0.064	0.042	1.53	1.53
Random	16	2	1.609	1.718	0.898	1.91	1.91
Random	32	4	7.744	3.780	1.830	2.06	2.06
Random	64	8	33.641	16.178	4.680	3.46	3.46
Random	96	12	72.337	19.357	5.638	3.43	3.43
Random	128	16	114.128	26.283	6.799	3.87	3.86
Random	192	24	397.060	66.641	17.646	3.78	3.78
Parallel	96	12	150.846	21.071	4.397	4.79	4.79
One matching	96	12	111.746	41.916	20.502	2.04	2.04
Filled	256	8	251.238	125.070	79.343	1.58	1.50

D, F and S denote maintained dense, array-factorized and positive-support sparse methods. The main

added-step comparison is F/S: random-fixture wall ratios are 1.91–3.87, the parallel fixture gives 4.79, and the $Q = 8$, $M = 256$ filled control gives 1.58 (1.50 CPU). The latter finishes with all 64 output cells positive, so final sparsity is not being assumed. Support sizes vary during the computation.

All eleven selected fixtures favor S in this run; this is not a universal crossover claim. Regressions in the earlier D/F step remain visible: F is slower than D on the one-edge $Q = 1$ fixture (D/F = .965) and the 16-edge $Q = 2$ fixture (D/F = .936). The source-matched earlier two-method ledger is retained as a historical ablation, not pooled with these new samples or silently replaced to remove its adverse cases.

Ratios below one are retained regressions, not discarded warmups or failures. Tiny-case medians are particularly noisy on a shared executor. A production dispatcher could retain array-factorized or dense updates for relevant cases, but an unimplemented policy is not credited with measured performance. No component ratio is multiplied into a hypothetical SAT solve time.

10 A new mixed-root implementation of an inherited identity

The projective coefficient identity is *already present* in the inherited kernel-improvements report, including individual root orders $\beta_j + 1$. Its implementation handles exponents one and two. The newer round-three implementation uses one common odd order. Our new contribution is an independently authored mixed-order 3/5 implementation, extending practical coverage to mixed exponents two and four in one fixed quotient ring. This is not a new permanent formula or a new SAT exponent bound.

Exact identity and selection. Let $H(Z) = \prod_{r=1}^{R} \sum_j a_{rj} Z_j$ be homogeneous with $\sum_j \beta_j = R$. Set zero-target variables to zero and omit them, handling the empty product separately. Fix a minimum-target coordinate k to one. For every $j \neq k$, choose an available odd order $q_j > \max(\beta_j, \beta_k)$ and an orthogonal root ω_j. Then

$$[Z^\beta]H = \left(\prod_{j\neq k} q_j\right)^{-1} \sum_{0\le g_j<q_j} \left(\prod_{j\neq k} \omega_j^{-\beta_j g_j}\right) H(Z_k = 1, (Z_j = \omega_j^{g_j})_{j\neq k}).$$

Orthogonality retains a monomial only if $\alpha_j = \beta_j + q_j m_j$, $m_j \ge 0$, on every free coordinate. Homogeneity gives $\alpha_k = \beta_k - \sum q_j m_j$, forcing all $m_j = 0$. Every divisor is odd, so this formula needs no guard bits. For an available-order function $m(b)$ equal to the smallest available odd order above b, a minimum-target pivot minimizes $\prod_{j\neq k} m(\max(\beta_j, \beta_k))$: swapping a larger pivot for a smaller one leaves their exchanged factor equal and can only decrease all other factors.

For $\beta_j \in \{2, 4\}$, orders 3 and 5 give $P = \prod_j (\beta_j + 1)/(1 + \min_j \beta_j)$ evaluations. For example, $\beta = (2, 2, 4, 4)$ takes 75 mixed-root evaluations, versus 125 with common order five and 112 with the strongest inherited antipodal grid. This is an implementation of the existing individual-order theorem.

Ring and precision. All timed algorithms use the same ring

$$(\mathbb{Z}/2^p\mathbb{Z})[U, V]/(U^4 + U + 1, V^\ell),$$

with rank 4ℓ. The residue U is checked to have order fifteen. Classical precision-doubling Newton steps $x \leftarrow x - x(x^{15} - 1)/15$ lift it from k to $\min(2k, p)$ bits. Writing $e = x^{15} - 1$ gives $(1 + e)(1 - e/15)^{15} = 1 + O(e^2)$; only odd 15 is inverted. The old bits and root identity are checked at each step. The preserved bitwise routine is a regression oracle. There are $\lceil \log_2 p \rceil$ lifting rounds, not $p - 1$. The lifted root's fifth and third powers supply the order-three and order-five roots. The program checks their

full orthogonality sums, not merely multiplicative order. The original source degree $e = 4\lceil \log_2 s \rceil$ already contains the required residue subfield, but representing and lifting its roots must still be paid. Our prototype does not silently enlarge the source ring or implement arbitrary source embeddings. Coefficients are canonicalized at immutable creation precision p, and the result is returned at p.

Streaming and matched comparisons. A common primitive fifteenth root permits a single integer phase index modulo fifteen. Mixed-radix Gray traversal updates the R row sums after one coordinate changes. With precomputed neighboring-root increments, work is $O(RP + R\sum_j q_j)$ ring operations and storage is $O(R\sum_j q_j + R + t)$ ring elements. No grid-indexed table is stored. The common-root comparator shares the same arithmetic, validation and Gray kernel. The antipodal comparator is *anisotropic*: step one for even targets and two for odd targets. Its guard loss is $v_2(\prod \beta_j!) + \sum_{\beta_j \text{ odd}} \beta_j - 1$, not the larger uniform step-two loss. Existing sign/cube kernels remain in the portfolio; the demonstration dispatcher does not claim to supersede them.

Exact checks. Normal and optimized Python both pass 1,880 exact coefficient equalities, 96 affine-adapter equalities, 83,696 independent monomial alias checks, 125 Gray-grid checks, 12 doubling-versus-bitwise root fixtures and seven rejection controls. Coverage includes noncanonical and negative coordinates, precision one, nilpotents, zero and singleton targets, mixed odd targets, and exhaustive small binary matrices. These are component regression tests, not a formal proof or a complete SAT run. The imported programs were not executed.

Target profile	p/ℓ	Common	Half	Mixed	C_f/M_f	H/M_f
$(2,2,4,4)$	32/1	125	112	75	1.847	1.005
$(2,2,2,4,4)$	64/1	625	337	225	2.421	1.531
$(2,2,2,2,4,4)$	128/1	3125	1012	675	4.672	1.386
$(2,2,4,4,4)$	32/1	625	562	375	1.662	1.560
$(2,2,4,4)$	32/2	125	112	75	1.591	1.102
Eight ones	32/1	2187	128	2187	1.162	0.042
$(4,4,4)$	32/1	25	62	25	1.054	1.728
$(2,2,4,4)$	1/1	125	112	75	1.338	1.677

The middle columns count evaluations. C_f/M_f and H/M_f are median wall-time ratios with *fresh root construction charged in every root-filter call*. Five rotated/reversed repetitions, one warmup per method, identical input and quotient ring, and full canonicalization, root validation, setup and extraction are included. The independent oracle and input generation are excluded. Raw wall/CPU samples and exact source/script hashes are retained. The six mixed-even fixtures show 1.34–4.67 times improvement versus fresh common-root filtering and 1.005–1.68 versus the strong half-grid comparator; the first fixture is effectively tied in these noisy measurements. Cached-root ratios are 1.54–6.26 and 1.27–2.29, respectively. These are selected synthetic components, not SAT workloads.

The adverse eight-ones profile is about 23 times slower with a cached root and 24 times slower with a fresh root; the dispatcher selects the half-grid. Equal-count all-four variants have no mixed-radix structural advantage. Some fresh-root medians are smaller than cached-root medians, reflecting shared-host variation, not negative overhead. Historical bitwise-root runs are retained, including a 3.4% nilpotent regression and fresh-root losses. They are not the final default's table. Count-only dispatch need not minimize wall time. No statistical significance, universal speedup or complete-SAT improvement is inferred. The robust claims are exact outputs, smaller grids on the specified profiles, polynomial storage and logarithmically many classical root-lifting rounds.

11 Barriers and unresolved interfaces

The strongest new results are constant improvements and component implementation changes. Several tempting routes to a smaller logarithmic order remain blocked by specific mathematical obligations. These are useful negative results, not claims that no other algorithm can do better.

11.1 Independent-jet Hermite recovery

The rectangular packet extends its Hermite optimization to arbitrary asymmetric node multiplicities. Inactive-node extension makes the independent-jet precision objective convex; averaging residue-tree automorphisms fixing zero gives a valuation-symmetric optimum. Backward rational recurrences compute the finite real optimum, and componentwise ceilings increase the normalized objective by less than two. Within this fixed independent-error model, the leading six survives arbitrary multiplicities.

That is a model-specific barrier. It does not rule out exploiting correlations among jets actually generated by the contraction, nor establish that every abstract admissible error pattern is attainable. A proposed improvement must exhibit and prove such additional structure rather than merely redistribute independent precision budgets.

11.2 Explicit tables and balanced trees

The table-width arguments apply to full boundary tables on complete pair graphs or the relaxed Euclidean entropy ball. A fixed tree has a balanced node, leading to a certificate width at least $2K/3$. Adaptive trees still have a uniform-positive-vector witness, with lower bound

$$\frac{4K}{9\sqrt{1-1/r}}.$$

The related cardinality statement $2\lceil r/3\rceil\,(r-\lceil r/3\rceil)$ concerns explicit full tables. These observations constrain the representation being optimized, not SAT itself, implicit contractions, cancellation-based methods or every feasible tensor family. An abstract equal-inventory witness is not proof that each SAT tensor realizes its extremal nonzero entries.

The four-plan success does not contradict these barriers: it improves the upper certificate within a finite menu and leaves the logarithmic scale unchanged. It also does not prove that four plans are best. Larger portfolios, different unbalanced trees or new backends might improve constants, but the finite costs of every table and recursive entry must remain included.

11.3 Precision peeling and degree flow

Discarding only a polylogarithmically thin budget tail retains the same leading entropy and inherited precision exponent. No free reset of cumulative precision follows from exposing a small residue or from a mode that currently evaluates to zero. In the Cauchy source recurrence, a zero displayed residue can later recover after normalization; it is unsound to terminate based only on that observation.

The source degree-flow identity is a valid algebraic invariant, but it does not supply the missing compiler. To obtain a genuinely smaller logarithmic order by recursion, one needs a proved interface simultaneously preserving low degree, compressed mode multiplicity, bounded local graph degree, short immutable descriptions and affordable recursive entry evaluation. None of the new coefficient schedules or public release automatically provides that package. This is the exact structural open loop rather than a vaguely missing optimization.

11.4 Practical integration boundary

The fixed rank-four mixed-root prototype does not find embeddings in every upstream source ring. The fast rectangular backend is not implemented end to end. The matching factorization is not the asymmetric labeler of the sharpest contraction theorem. Consequently, a complete contraction harness must still choose compatible branches, track creation precision, materialize only justified features, and account for all conversions and output requirements.

The next useful experiment is not a larger isolated speedup against an obsolete routine. It is an exact component-to-contraction integration with instrumented entry counts, bit lengths, cache sizes and preparation costs. Its output should reveal how much total time is spent in the optimized component and whether the hypotheses used in the asymptotic schedule are maintained. That would be new evidence; it is not inferred from the present synthetic tables.

12 Evidence levels and independent checks

The evidence separates mathematical derivations relative to named imports, exact finite certificates, source-hashed synthetic measurements, and published external Lean/nanoda verification. None is silently promoted to another: finite tests do not prove all-size runtime bounds, timings do not establish SAT performance, and verification of the independent formal development does not cover our new refinements.

12.1 Independent implementation audit

The second implementation audit does not reuse the production coefficient dynamic program. Its oracle convolves sparse polynomials over unrestricted integers, reduces $U^4 = -U - 1$ and $V^\ell = 0$, and only then reduces coordinates modulo 2^p. Its affine oracle multiplies affine forms directly rather than reproducing the homogenization adapter. Its matching oracle enumerates all complete labelings and averages final exponential-load potentials under prefixes, independently reconstructing candidate values and ties.

The final audit passes 180 integer quotient products, 18 high-precision root/property comparisons through 257 bits, 176 homogeneous outputs, 214 affine outputs, seven proper-order root rejections and 1,364 available-order pivot profiles. Its 126 matching instances enumerate 2,400 complete labelings and check 744 conditional candidate values. It recomputes the medians and available source/script hashes for eight mixed-kernel and eleven matching fixtures. The same checks pass under `python -O`. On the saturated matching control, an independent state observer reconstructs all 512 vertex steps, including 440 full-support and 72 absent-support choices, and verifies all 64 final cells have load four.

The schedule audit independently encloses the degree-.51 coefficient with a different arctangent identity. The independent four-plan audit uses a different operation-set construction and tuple order. These are meaningful cross-checks of new finite components; they are not an assertion that two programs constitute a proof assistant or that all shared imported mathematics has been independently checked.

12.2 Changes prompted by review

Review strengthened the dense matching comparator with cached row sums, corrected the mixed-root novelty claim, and made both fresh-root methods pay the same precision-doubling construction. The generic four-plan time bound includes its dominant paid tail, and the low-degree taper was retuned to

pay descendants. Public proof status was corrected without inventing a chronology that would excuse the earlier omission. These changes are reflected in the final proofs and source-matched ledgers, not hidden behind obsolete headline measurements.

13 Reproducibility and artifact map

The report is standalone LaTeX source with a compiled, visually reviewed PDF. The surrounding bundle separates authenticated incoming evidence, newly written implementations, exact checkers, raw measurements and review notes. The intake archive identities are in `incoming/manifest.json`; final tree-file equivalence is in `closing_identity_checks.json` and `closing_git_inventory.json`. The closing pin is `7de9d09fecd17de776d712835ecb2011205c531d`. The public formalization has its own immutable source archive and API/log evidence under `anthropic_snapshot/`.

File or directory	Purpose
`four_plan_portfolio_new.md`	Final generic four-plan proof and exact menu
`rectangular_tree_new.md`	Three-plan ablation and full schedule gates
`matrix_checks/verify_four_plan.py`	Standard-library exhaustive final certificate
`tree_checks/audit_four_plan.py`	Independently structured quadruple audit
`tree_checks/check_schedule_combinations.py`	Streamed and low-degree exact sums
`tree_checks/matching_factorized.py`	Dense, factorized and positive-support matching
`code/mixed_projective.py`	Mixed-root, common-root and antipodal kernels
`qa/check_components_independent.py`	Independent integer and completion oracles
`qa/check_degree051_independent.py`	Independent intermediate-degree certificate
`results/` and `tree_checks/`	Source-hashed raw timing and check receipts
`anthropic_revision_review.md`	Exact public theorem and CI assessment
`anthropic_snapshot/`	Pinned source, tree equivalence, log and inventory

13.1 Replaying exact checks

From a fresh copy of the bundle root, with a compatible Python 3 interpreter, representative commands are:

```
python3 matrix_checks/verify_rectangular_tree.py
python3 matrix_checks/verify_four_plan.py
python3 matrix_checks/verify_degree_051_cached.py
python3 -O matrix_checks/verify_four_plan.py
python3 tree_checks/audit_four_plan.py
python3 tree_checks/check_schedule_combinations.py
python3 qa/check_degree051_independent.py
python3 code/check_mixed_projective.py
python3 -O code/check_mixed_projective.py
python3 tree_checks/check_matching_factorized.py
python3 -O tree_checks/check_matching_factorized.py
python3 qa/check_components_independent.py
python3 -O qa/check_components_independent.py
```

The final support replay uses only the standard library and imports only the newly authored checker beside it. The exploratory NumPy scripts are optional and are not required for the final certificate. No installation or execution of the incoming archives is needed to replay the new finite work.

For timing replay, use fresh output names and preserve the delivered historical ledgers:

```
python3 code/benchmark_mixed_projective.py \
  --repetitions 5 --output results/mixed_replay.json
python3 tree_checks/benchmark_matching_factorized.py \
  > tree_checks/matching_replay.json
```

Some exact check scripts print JSON; others also write a standard receipt. Replay in a copy when historical bytes must remain untouched. Wall times will differ across hosts. Equality, deterministic counts, exact certificates and source identity are the portable claims; historical medians are observations, not required outputs.

13.2 Fair comparison protocol

For matching, keep the same graph, parallel-edge treatment, first-fit coloring, vertex order, rational parameter, smallest-label tie rule, selected trace and dense output. Charge each method's actual invariant checks, and distinguish symbolic candidate averaging from a requested full candidate output. Do not compare a preinitialized sparse routine with a dense routine paying initialization. Keep the row-sum optimization in the dense comparator.

For extraction, keep the quotient ring, nilpotent length, target profile, immutable input representatives, output precision and setup policy identical. Use the anisotropic guard loss, and distinguish cached roots from fresh roots. Common and mixed fresh-root paths must pay the same lifting method. A lower evaluation count does not imply minimum wall time when field operations, precision and validation differ.

For theoretical schedules, replay all child-call prefixes, output branches, tail limits, degree gates and stopping thresholds. A smaller local coefficient is not a complete resource theorem until these terms are paid. Archive text and stored check receipts are provenance, not substitutes for an independently reviewed proof or newly executed exact certificate.

14 Conclusions

The strongest final generic certificate in this review is the explicit four-plan rectangular composition: safe eventual time 30.492 and space 28.854 on the normalized $E(s)$ scale. Polynomial-space generic time is safely 46.072. Under degree $s^{\cdot 51}$ it is 21.14 in polynomial space or 14.503 in cached time and space; under degree $s^{1/12}$ the cached time and space are both safely 12.016. These are improvements against the updated compatible incoming frontier, with earlier three-plan results retained as matched ablations.

The matching factorization gives a genuine update-order reduction and a further positive-support implementation, while the mixed-root work supplies broader implementation coverage for a known identity and a standard, better root-preparation schedule. Exact outputs, carefully strengthened comparators and adverse cases support these component claims. They do not support a universal practical crossover or an end-to-end SAT speedup.

The independent Anthropic release now has public source and exact-tree published Lean/nanoda acceptance. That is a substantial proof-status change and deserves a direct correction of the older description. It does not alter the scope of the new normalized refinements, erase semantic auditing duties or supply a numerical performance comparison between the two independent architectures.

A smaller logarithmic order remains a separate research problem. It requires new structural control of recovery, degree, representation or summation, not another unqualified decimal constant. The present bundle makes the positive results reproducible while stating that boundary explicitly.

A Source inventory and citations

The following short names identify the reviewed incoming archives. The archived previous iteration is counted as provenance, not a new independent proposal. Full byte counts, Git blobs and SHA-256 values are in the intake manifest; exact final-path equivalence is separately recorded.

1. `3sat_round3_frontier_20261027.zip`
2. `ETH_Adaptive_Exact_Kernels_2026-10-27.zip`
3. `eth_adaptive_schedules_2026-10-26.zip`
4. `sat_adaptive_bounds_2026-10-26.zip`
5. `sat_adaptive_frontier_2026-10-26.zip`
6. `sat_adaptive_matrix_partition_2026-10-26.zip`
7. `sat_adaptive_tree_certificate_2026-10-26.zip`
8. `subexponential_sat_adaptive_four_2026-10-26.zip`
9. `subsat_rectangular_2026-10-26.zip`

The mathematical citations below point to primary public sources. All statements about the new iteration's exact results are additionally tied to the authored proof and receipt files identified in Section 13.

References

[1] Vladimir Reshetnikov, *ProveIt: SubexponentialSAT research corpus.* Intake commit `73dfb5e5ab6e91bd7375e5b97ef85d7e1cc8e81d`; closing commit `7de9d09fecd17de776d712835ecb2011205c531d`.
Pinned project research tree.

[2] *New Bounds for Matrix Multiplication: from Alpha to Omega*, arXiv:2307.07970. The conservative inequalities used here are $\alpha > .3$ and $\omega < 19/8$, not a claim about the current best record.
Primary paper and versions.

[3] Anthropic, *Add eth: paper and Lean 4 formalization of A* $2^{o(n)}$ *Time Algorithm for 3-SAT*, PR #31, requested head `1a6eee293b9e00cb432eef6018a21f523623a9af`.
Public pull request; requested commit.

[4] Anthropic, *ETH main formal statements and solution*, pinned revision.
MainResults.lean; Solution.lean; project README.

[5] Anthropic repository verification, *verify (eth)*, run 40440389861, job 121979112896, completed successfully 27 October 2026.
Published job; Comparator configuration; verification script.

[6] Claude / Anthropic, *A* $2^{o(n)}$ *Time Algorithm for 3-SAT*, 26 October 2026, 49-page released paper.
Pinned paper; formal-to-paper index.